Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

CompTIA Certified Server+ Training

TGS Ref No: TGS-2024048318

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 4.1

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
3.0	1 May 2025	Previous release — CompTIA Certified Server+ (SK0-005) master trainer slides and learner guide.	Tertiary Infotech Academy
4.0	19 August 2026	Rebuilt courseware from the single-source pipeline: Learner Guide with step-by-step guides for all 33 hands-on Killercoda labs across the four SK0-005 exam domains (18/30/24/28%), plus an Exam Focus section on RAID formulas, subnetting, security and the troubleshooting methodology.	Dr Alfred Ang
4.1	19 August 2026	Restructured each lab into its own folder with guidelines/cheatsheet/worksheet PDFs and CSV+Excel mock data; added Linux/Kubernetes/Docker lab environments and bonus container/Kubernetes assets (Dockerfiles, Compose, Kubernetes YAML, shell scripts).	Dr Alfred Ang

TABLE OF CONTENTS

Introduction

This Learner Guide accompanies the WSQ course CompTIA Certified Server+ Training (TGS-2024048318), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 33 hands-on labs, organised by the four official CompTIA Server+ SK0-005 exam domains. Every lab maps to a published exam objective and is completed on the free Killercoda Ubuntu playground using standard, free server administration tools.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. Where an exam objective is about physical kit (racking, PSUs, hot-swap bays, BIOS/UEFI screens), the labs use the closest software equivalents available on Linux — loopback disks for RAID, virtual NICs, KVM/QEMU virtual machines, and inspection tools such as dmidecode, ipmitool and smartctl — so every concept can be reproduced inside the disposable VM. Labs that scan, test passwords, simulate failures or destroy media must only be run against systems you own or are authorised to test; unauthorised access is an offence under the Singapore Computer Misuse Act.

Course Learning Outcomes

- LO1: Install and manage server hardware — racking, cabling, power and cooling; deploy and manage storage (RAID, shared storage, capacity planning); and maintain hardware with out-of-band management and firmware upgrades.
- LO2: Administer server operating systems and services — OS installation, partitions and file systems, IP/DNS/DHCP/VLANs, server roles, monitoring, high availability, virtualization, scripting and asset management.
- LO3: Secure servers and plan for disaster recovery — data and physical security, identity and access management, mitigation strategies, server hardening, decommissioning, backup and recovery.
- LO4: Troubleshoot server hardware, software, storage, OS and network problems using a structured methodology, and validate disaster-recovery and failover readiness.

Before You Start — Environment Setup

Lab environments

The 33 core labs run on the free Killercoda Ubuntu (Linux) playground. A few labs also ship bonus container and Kubernetes assets that run on Docker Desktop or the Killercoda Kubernetes playground:

- Linux (all core labs) — Killercoda Ubuntu playground: https://killercoda.com/playgrounds/scenario/ubuntu
- Kubernetes (bonus assets, Labs 14 & 15) — Killercoda Kubernetes playground: https://killercoda.com/playgrounds/scenario/kubernetes

- Docker (bonus assets, Labs 10 & 15) — Docker Desktop: https://www.docker.com/products/docker-desktop/

Each lab is its own folder in labs/ (e.g. labs/lab-02-raid-mdadm/) containing the guide (.md), a printable guidelines PDF, a cheatsheet PDF and a fill-in worksheet PDF, plus CSV+Excel mock data for data-bearing labs and Dockerfiles / Kubernetes YAML / shell scripts where relevant.

What you need

- A web browser — every core lab runs on the free Killercoda Ubuntu playground: https://killercoda.com/playgrounds/scenario/ubuntu
- No local install or physical hardware is required. Each package is pulled with apt (or a single binary download) inside the throw-away VM.
- For the bonus container labs: Docker Desktop (https://www.docker.com/products/docker-desktop/) and the Killercoda Kubernetes playground.
- Reset the playground between labs that change storage, firewall, RAID or service state, so each lab starts clean.
- Optional free online helpers: the IP Calculator (https://alfredang.github.io/ipcalculator/) for subnet planning, and the SSL Labs test for TLS posture.

Launch and verify the playground

Open the Killercoda Ubuntu playground in your browser. You start as root in a full Ubuntu VM. Confirm the VM is ready and refresh the package lists before you begin — then install each lab's tools with apt as the lab directs.

```
root@playground:~# id                # confirm you are root

root@playground:~# uname -a          # confirm the Ubuntu VM

root@playground:~# apt update        # refresh package lists

root@playground:~# apt install -y mdadm lvm2   # example: install a lab's
tools
```

Conventions used in every lab

- Commands are run in the Killercoda VM as root (the playground logs you in as root).
- Placeholders such as <disk>, <ip> and <host> are replaced with the values shown in each lab.
- Loopback files (/dev/loopN) stand in for physical disks; virtual NICs and KVM VMs stand in for physical kit.
- Reset the playground between labs that change storage, firewall, RAID or service state.
- Follow the lab's clean-up notes so a later lab is not affected by an earlier one.

Topic 01 — Server Hardware Installation and Management (18%)

Racking & power · storage & RAID · out-of-band management & firmware

Key concepts

- Form factors & rackingRack (1U/2U/4U) vs. tower vs. blade; rack units, rail kits, cable management, weight distribution and airflow direction (front-to-back).
- Power & coolingRedundant PSUs, dual power feeds, PDUs and UPS; hot vs. cold aisle containment; environmental targets for temperature and humidity.
- RAID levelsRAID 0/1/5/6/10 and JBOD — capacity and fault-tolerance formulas; hardware vs. software RAID; when RAID 6 or RAID 10 wins.
- Shared storageDAS vs. NAS vs. SAN; NFS/SMB (file) vs. iSCSI/Fibre Channel (block); LUNs, and capacity planning with growth headroom.
- Out-of-band managementIPMI/BMC, iDRAC/iLO for lights-out control; firmware/BIOS/UEFI updates and the update order.
- Hot-swap & maintenanceHot-swappable drives, PSUs and fans; hot-add vs. cold maintenance; component replacement without downtime.

Lab 1 — Server Form Factors, Racking, and Power Planning

Exam objective: 1.1 Install physical hardware.

Goal: The learner inventories a running system with dmidecode/lshw and plans a 42U rack layout with power and cooling rules the way a data-centre technician would.

What you'll build

A rack layout plan plus a hardware inventory and power-budget sanity check. (Tools: dmidecode, lshw, lscpu, lspci, lsusb.)

Step-by-step

1. Install the hardware inventory tools

```
apt update && apt install -y dmidecode lshw pciutils usbutils
```

2. Identify the running system as a rack-mount server

```
dmidecode -t system | grep -E "Manufacturer|Product|Serial"
```

3. Read chassis type and U-height

```
dmidecode -t chassis | grep -E "Type|Height|Power"
```

4. Inventory CPU, RAM, bus types and expansion cards

```
lscpu | head -20
```

5. List PCI/PCIe buses and cards

```
lspci | head
```

6. Get the model string for a hardware compatibility list (HCL) check

```
dmidecode -s system-product-name
```

7. Plan a 42U rack layout: heaviest at bottom, redundant power on separate circuits
8. Do a power-budget sanity check against the PDU and breaker (80% rule)

Test it

The learner verifies success by reading chassis/CPU/memory data and producing a rack layout that respects U sizing, weight balancing, cooling and redundant power.

Note: Full commands and reference links are in labs/lab-01-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 2 — Storage Deployment: RAID 0/1/5/6/10 with mdadm

Exam objective: 1.2 Deploy and manage storage (RAID levels, JBOD, capacity planning).

Goal: The learner builds every RAID level the exam tests on loopback files with mdadm, then fails and rebuilds an array to observe resync.

What you'll build

Working RAID 0/1/5/6/10 and JBOD arrays plus a completed rebuild. (Tools: mdadm, losetup, mkfs.ext4.)

Step-by-step

9. Install mdadm

```
apt update && apt install -y mdadm
```

10. Create six loopback disks that behave like drives

```
for i in 1 2 3 4 5 6; do truncate -s 200M disk$i.img; losetup /dev/loop$i
disk$i.img; done
```

11. Build a RAID 0 stripe (N x disk, no redundancy)

```
mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/loop1 /dev/loop2
```

12. Build a RAID 1 mirror (survives 1 disk failure)

```
mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/loop1 /dev/loop2
```

13. Build a RAID 5 single-parity array

```
mdadm --create /dev/md5 --level=5 --raid-devices=3 /dev/loop1 /dev/loop2
/dev/loop3
```

14. Build a RAID 6 double-parity array

```
mdadm --create /dev/md6 --level=6 --raid-devices=4 /dev/loop1 /dev/loop2
/dev/loop3 /dev/loop4
```

15. Build a RAID 10 stripe of mirrors

```
mdadm --create /dev/md10 --level=10 --raid-devices=4 /dev/loop1
/dev/loop2 /dev/loop3 /dev/loop4
```

16. Simulate a failure and rebuild, watching resync

```
mdadm /dev/md1 --fail /dev/loop2 --remove /dev/loop2
```

17. Compare hardware RAID vs software RAID trade-offs

Test it

The learner verifies success by reading the array status in /proc/mdstat and confirming the capacity and fault-tolerance formula for each level.

Note: Full commands and reference links are in labs/lab-02-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 3 — LVM Provisioning and Capacity Planning

Exam objective: 1.2/2.1/2.3 Capacity planning and logical volume management.

Goal: The learner provisions storage through the PV to VG to LV model, extends a volume online, adds swap and takes a point-in-time snapshot.

What you'll build

An LVM volume group with web, database and swap volumes plus a live snapshot. (Tools: lvm2, mkfs.ext4, mkfs.xfs.)

Step-by-step

18. Install lvm2

```
apt update && apt install -y lvm2
```

19. Create four loopback disks

```
for i in 1 2 3 4; do truncate -s 500M disk$i.img; losetup /dev/loop$i
disk$i.img; done
```

20. Create physical volumes and a volume group

```
vgcreate vg_data /dev/loop1 /dev/loop2 /dev/loop3
```

21. Create logical volumes for web and database

```
lvcreate -L 800M -n lv_web vg_data
```

22. Extend the DB volume online (add PV, grow LV, grow FS)

```
lvextend -L +400M /dev/vg_data/lv_db
```

23. Grow the XFS filesystem online

```
xfs_growfs /srv/db
```

24. Create a dedicated swap LV

```
lvcreate -L 200M -n lv_swap vg_data
```

25. Take a point-in-time snapshot of a live volume

```
lvcreate -L 100M -s -n lv_web_snap /dev/vg_data/lv_web
```

26. Report capacity vs utilisation as a baseline

```
vgs --units g -o vg_name,vg_size,vg_free
```

Test it

The learner verifies success by confirming the extended volume grew online and the snapshot restores a deleted file.

Note: Full commands and reference links are in labs/lab-03-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 4 — Shared Storage: NAS (NFS/CIFS) and SAN (iSCSI)

Exam objective: 1.2 Shared storage: NAS (NFS, CIFS), SAN (iSCSI).

Goal: The learner exports file-level NFS and CIFS shares and builds a block-level iSCSI target, then connects an initiator and formats the LUN as a disk.

What you'll build

A working NFS share, a Samba/CIFS share and a mounted iSCSI SAN LUN. (Tools: nfs-kernel-server, samba, tgt, open-iscsi.)

Step-by-step

27. Install the NAS and SAN packages

```
apt update && apt install -y nfs-kernel-server nfs-common tgt open-iscsi
samba
```

28. Export an NFS share (file-level NAS)

```
echo "/srv/nfs/share  127.0.0.1(rw,sync,no_subtree_check,no_root_squash)"
>> /etc/exports
```

29. Mount the NFS share as a client

```
mount -t nfs 127.0.0.1:/srv/nfs/share /mnt/nas
```

30. Create a CIFS/SMB share and browse it

```
smbclient -L //127.0.0.1 -N
```

31. Build an iSCSI target with a 500 MB LUN

```
systemctl restart tgt
```

32. Discover and log in to the iSCSI target

```
iscsiadm -m discovery -t st -p 127.0.0.1
```

33. Log in to the iSCSI node

```
iscsiadm -m node --login
```

34. Format and mount the new SAN LUN as a local disk

```
mkfs.ext4 /dev/$NEW
```

35. Compare NAS vs SAN vs Fibre Channel/FCoE

Test it

The learner verifies success by reading files over the NFS/CIFS mounts and confirming a new /dev/sdX LUN appears from the iSCSI target and mounts.

Note: Full commands and reference links are in labs/lab-04-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 5 — Out-of-Band Management, BIOS/UEFI, and Firmware

Exam objective: 1.3 Perform server hardware maintenance (OOB, BIOS/UEFI, firmware, hot-swap).

Goal: The learner inspects BIOS/UEFI from the OS, practises the efibootmgr and ipmitool workflows, and walks the fwupd firmware-upgrade and hot-swap checklists.

What you'll build

A firmware/BIOS inventory and a documented OOB management and hot-swap workflow. (Tools: ipmitool, fwupd, efibootmgr, dmidecode.)

Step-by-step

36. Install the OOB and firmware tools

```
apt update && apt install -y ipmitool fwupd efibootmgr dmidecode
```

37. Inventory BIOS/UEFI from the running OS

```
dmidecode -t bios
```

38. Confirm UEFI vs legacy BIOS boot mode

```
[ -d /sys/firmware/efi ] && echo "Booted in UEFI mode" || echo "Booted in
legacy BIOS mode"
```

39. List UEFI boot entries

```
efibootmgr -v
```

40. Practise the IPMI/BMC chassis-status pattern

```
ipmitool -H bmc.example.com -U admin -P 'CHANGEME' chassis status
```

41. List which firmware is installed with fwupd

```
fwupdmgr get-devices
```

42. Check for available firmware updates

```
fwupdmgr get-updates
```

43. Walk the four local-admin paths (crash cart, KVM, serial, virtual console)
44. Walk the hot-swap pre-swap checklist for drives, PSUs and fans

Test it

The learner verifies success by reading the BIOS version and boot mode and mapping each ipmitool command to its Server+ OOB sub-objective.

Note: Full commands and reference links are in labs/lab-05-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Topic 02 — Server Administration (30%)

OS install & file systems · networking · roles, HA, virtualization, scripting & assets

Key concepts

- OS installationAttended, unattended, scripted and image-based installs; partition schemes (MBR vs. GPT); file systems (ext4, XFS, ZFS, NTFS).
- Network servicesIPv4/IPv6 addressing and subnetting, DNS, DHCP, FQDN, VLANs and default gateways for a server.
- Server roles & monitoringWeb, file, database, directory and print roles; performance baselining, metrics and thresholds; data migration.
- High availabilityClustering, load balancing, NIC teaming and failover to remove single points of failure.
- Virtualization & cloudHost vs. guest, hypervisor types, resource allocation and over-commit; IaaS/PaaS/SaaS and public/private/hybrid models.
- Scripting & asset managementVariables, loops and conditionals for common server tasks; documentation, lifecycle and secure asset storage.

Lab 6 — Installing a Server OS (Unattended / Scripted)

Exam objective: 2.1 Install server operating systems.

Goal: The learner validates minimum specs and the HCL, builds a cloud-init autoinstall seed.iso, captures a golden image and walks the P2V conversion workflow.

What you'll build

A cloud-init autoinstall seed.iso and a golden-image tarball for template deployment. (Tools: debootstrap, cloud-init, genisoimage, qemu-utils.)

Step-by-step

45. Install the OS-deployment tools

```
apt update && apt install -y debootstrap cloud-init genisoimage qemu-utils
```

46. Confirm minimum OS requirements (CPU, RAM, disk)

```
lscpu | grep -E "Architecture|CPU\(s\)"
```

47. Do an HCL check on CPU, NIC and storage controller

```
echo "NIC: $(lspci | grep -i ether)"
```

48. Build a minimal cloud-init autoinstall config
49. Package the config into a NoCloud seed ISO

```
genisoimage -output seed.iso -volid cidata -joliet -rock user-data meta-
data
```

50. Capture a golden image of the root filesystem

```
tar --one-file-system --exclude=/proc --exclude=/sys --exclude=/dev --
exclude=/mnt -czf /srv/golden.tgz / 2>/dev/null
```

51. Build a template root filesystem with debootstrap

```
debootstrap --variant=minbase jammy /srv/template
http://archive.ubuntu.com/ubuntu/
```

52. Create a destination image shape for P2V conversion

```
qemu-img create -f qcow2 /srv/converted.qcow2 5G
```

53. Compare GUI vs Core vs Bare metal vs Virtualized vs Remote installs

Test it

The learner verifies success by confirming the seed.iso is built and the golden-image tarball and debootstrap template are deployable filesystems.

Note: Full commands and reference links are in labs/lab-06-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 7 — Partition and Volume Types, File Systems

Exam objective: 2.1 Partition/volume types (GPT vs MBR, LVM) and file systems.

Goal: The learner partitions a disk with MBR then GPT, formats ext4/XFS/ZFS, persists mounts by UUID and converts MBR to GPT non-destructively.

What you'll build

A GPT-partitioned disk with ext4, XFS and ZFS filesystems and a UUID-based fstab entry. (Tools: parted, gdisk, e2fsprogs, xfsprogs, zfsutils-linux.)

Step-by-step

54. Install the partitioning and filesystem tools

```
apt update && apt install -y parted gdisk e2fsprogs xfsprogs zfsutils-linux
ntfs-3g
```

55. Create a 4G loopback disk to partition

```
losetup -P /dev/loop10 /srv/disk.img
```

56. Create an MBR (msdos) partition table

```
parted -s /dev/loop10 mklabel msdos
```

57. Switch the disk to a GPT partition table

```
parted -s /dev/loop10 mklabel gpt
```

58. Format the ext4 partition

```
mkfs.ext4 /dev/loop10p1
```

59. Create a ZFS pool on a partition

```
zpool create -f tank /dev/loop10p3
```

60. Compare the five Server+ filesystems (ext4, NTFS, VMFS, ReFS, ZFS)
61. Persist a mount by UUID in /etc/fstab

```
echo "UUID=$UUID  /mnt/p1  ext4  defaults,noatime  0 2" | tee -a /etc/fstab
```

62. Convert MBR to GPT non-destructively with gdisk

```
gdisk -l /dev/loop10 | head
```

Test it

The learner verifies success by printing the partition table with parted/gdisk and confirming the UUID-based fstab mount survives mount -a.

Note: Full commands and reference links are in labs/lab-07-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 8 — IP, VLAN, DNS, DHCP, FQDN, and Hosts File

Exam objective: 2.2 Configure servers to use network infrastructure services.

Goal: The learner configures a static IP and gateway, tags a VLAN, builds a BIND9 zone with A records and a DHCP scope, and simulates APIPA.

What you'll build

A configured static IP with VLAN, a BIND9 zone serving an FQDN and a validated DHCP scope. (Tools: iproute2, bind9, dnsutils, isc-dhcp-server.)

Step-by-step

63. Install the networking service packages

```
apt update && apt install -y iproute2 vlan bind9 bind9utils dnsutils isc-dhcp-server
```

64. Inspect IP configuration, routes and MAC addresses

```
ip -c addr
```

65. Configure a static IP and default gateway

```
ip addr add 10.99.99.10/24 dev eth0
```

66. Tag VLAN 20 on a single NIC with 802.1Q

```
ip link add link eth0 name eth0.20 type vlan id 20
```

67. Review RFC 1918 private ranges and APIPA
68. Build a BIND9 zone with A records and restart

```
systemctl restart bind9
```

69. Resolve the FQDN against the local server

```
dig @127.0.0.1 web.lab.local +short
```

70. Add a hosts-file override that beats DNS

```
echo "10.99.99.50  app.lab.local app" >> /etc/hosts
```

71. Validate an ISC DHCP scope config

```
dhcpd -t -cf /etc/dhcp/dhcpd.conf && echo "Config OK"
```

Test it

The learner verifies success by resolving web.lab.local through BIND9 and confirming the DHCP config passes validation.

Note: Full commands and reference links are in labs/lab-08-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 9 — Server Firewall and Port Management

Exam objective: 2.2 Firewall, Ports.

Goal: The learner memorises the well-known ports, applies a default-deny UFW policy, translates it to raw nftables and audits listening vs reachable vs permitted.

What you'll build

A default-deny firewall permitting only SSH/HTTP/HTTPS with source-restricted rules. (Tools: ufw, nftables, nginx, nmap.)

Step-by-step

72. Install the firewall and scan tools

```
apt update && apt install -y ufw nftables nginx nmap
```

73. Review the well-known server port table
74. Start a service and observe its open port

```
ss -tlnp | grep nginx
```

75. Apply a default-deny UFW policy with explicit allows

```
ufw default deny incoming
```

76. Allow SSH, HTTP and HTTPS

```
ufw allow 22/tcp comment 'SSH admin'
```

77. Confirm the policy with nmap

```
nmap -p 22,23,25,80,443,3306 127.0.0.1
```

78. Write the equivalent rule in raw nftables

```
nft add rule inet filter input tcp dport 8080 accept
```

79. Add a source-restricted rule for MySQL

```
ufw allow from 10.99.99.0/24 to any port 3306 proto tcp comment 'MySQL app
tier'
```

80. Run the 3-view audit: listening, reachable, permitted

```
ss -tulnp
```

Test it

The learner verifies success by confirming nmap sees only the allowed ports open and the ss, nmap and ufw views agree.

Note: Full commands and reference links are in labs/lab-09-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 10 — Server Roles: Web, File, and Database

Exam objective: 2.3 Server roles requirements (Web, File, Database).

Goal: The learner stands up nginx, Samba and MariaDB end-to-end on one host, applies resource limits and documents a role inventory.

What you'll build

A single host serving web, file and database roles with per-role resource limits. (Tools: nginx, samba, mariadb-server, curl.)

Step-by-step

81. Install the three role packages

```
apt update && apt install -y nginx samba mariadb-server curl cifs-utils
```

82. Start nginx and serve a page (web role)

```
curl -s http://127.0.0.1/ | head
```

83. Add a name-based virtual host

```
ln -sf /etc/nginx/sites-available/site1 /etc/nginx/sites-enabled/
```

84. Create a Samba user and share (file role)

```
(echo 'P@ssw0rd!'; echo 'P@ssw0rd!') | smbpasswd -s -a alice
```

85. Connect to the Samba share

```
smbclient //127.0.0.1/share -U alice%P@ssw0rd! -c 'ls'
```

86. Create a MariaDB database and least-privilege user (database role)

```
mysql -e "CREATE DATABASE appdb;"
```

87. Apply systemd resource limits so roles don't starve each other

```
systemctl set-property mariadb.service MemoryMax=512M CPUQuota=80%
```

88. Document the role inventory to a file

Test it

The learner verifies success by getting a page from nginx, listing the Samba share and running a SELECT against MariaDB.

Note: Full commands and reference links are in labs/lab-10-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 11 — Directory Services with OpenLDAP

Exam objective: 2.3/3.3 Directory connectivity, scope-based delegation.

Goal: The learner installs OpenLDAP, builds an OU/user/group tree, binds as a user and adds a group-based delegation ACL.

What you'll build

An OpenLDAP directory with an OU structure, users, groups and a delegation ACL. (Tools: slapd, ldap-utils, ldapscripts.)

Step-by-step

89. Install and preseed slapd non-interactively

```
DEBIAN_FRONTEND=noninteractive apt install -y slapd ldap-utils ldapscripts
```

90. Verify the directory is running

```
ldapsearch -x -H ldap://127.0.0.1 -b "dc=lab,dc=local" -s base
"(objectclass=*)" namingContexts
```

91. Create the people and groups OUs

```
ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/ou.ldif
```

92. Add user accounts

```
ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/users.ldif
```

93. Set a user password

```
ldappasswd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -s 'AlicePass!'
"uid=alice,ou=people,dc=lab,dc=local"
```

94. Create groups and assign membership

```
ldapadd -x -D "cn=admin,dc=lab,dc=local" -w LabAdmin1! -f /tmp/groups.ldif
```

95. Bind as a user and confirm identity

```
ldapwhoami -x -D "uid=alice,ou=people,dc=lab,dc=local" -w 'AlicePass!'
```

96. Add a group-based delegation ACL

```
ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/acl.ldif
```

97. Map the concepts to Active Directory (OU=scope, group=role, ACL=delegation)

Test it

The learner verifies success by binding as alice and searching entries under ou=people.

Note: Full commands and reference links are in labs/lab-11-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 12 — Performance Monitoring, Baselining, Event Logs

Exam objective: 2.3 Monitoring, baselining, event logs, thresholds, alerting.

Goal: The learner captures CPU/memory/disk/network baselines, inspects and rotates event logs, and wires threshold alerting with Prometheus node_exporter.

What you'll build

A saved performance baseline file and a node_exporter metrics endpoint for alerting. (Tools: sysstat, htop, iotop, fio, prometheus-node-exporter.)

Step-by-step

98. Install the monitoring tools and enable sysstat

```
apt update && apt install -y sysstat htop iotop fio prometheus-node-exporter
```

99. Read uptime and load averages

```
uptime
```

100. Capture a CPU baseline

```
mpstat 1 5
```

101. Capture a memory baseline

```
sar -r 1 5
```

102. Generate I/O load and measure IOPS and latency

```
iostat -xz 1 5
```

103. Save a combined baseline file for later diffing
104. Inspect event logs and configure rotation/retention

```
journalctl --since "1 hour ago" -p err
```

105. Expose metrics with node_exporter for thresholds/alerting

```
curl -s http://127.0.0.1:9100/metrics | grep -E 'node_cpu_seconds_total|node_memory_MemAvailable|node_filesystem_free' | head
```

Test it

The learner verifies success by producing a baseline file and confirming node_exporter serves the CPU, memory and filesystem metrics.

Note: Full commands and reference links are in labs/lab-12-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 13 — Data Migration and Transfer

Exam objective: 2.3 Data migration and transfer (rsync, SCP, Robocopy).

Goal: The learner transfers data with rsync, SCP and SFTP, applies fast-copy techniques and reviews exfiltration controls including bandwidth caps.

What you'll build

A mirrored dataset transferred with rsync/SCP/SFTP plus egress controls. (Tools: rsync, openssh-client, lftp, pv.)

Step-by-step

106. Install the transfer tools and start ssh

```
apt update && apt install -y rsync openssh-client openssh-server lftp pv
```

107. Set up source and destination trees

```
for i in $(seq 1 50); do dd if=/dev/urandom of=/srv/src/file$i.bin bs=1K
count=$((RANDOM%200+10)) 2>/dev/null; done
```

108. Mirror with rsync (archive, delta, delete)

```
rsync -avh /srv/src/ /srv/dst/
```

109. Rsync over SSH to a remote host

```
rsync -avzh -e ssh /srv/src/  user@remote:/srv/dst/
```

110. Copy with SCP (simple, no resume)

```
scp -r /srv/src/ root@127.0.0.1:/srv/dst-scp/
```

111. Transfer interactively over SFTP

```
sftp root@127.0.0.1
```

112. Compare cross-OS choices including Windows Robocopy
113. Fast-copy a large file showing throughput

```
pv /srv/src/file1.bin > /srv/dst/file1.bin
```

114. Apply a bandwidth cap to limit exfiltration

```
rsync -avh --bwlimit=1024 /srv/src/ user@remote:/srv/dst/
```

Test it

The learner verifies success by confirming the destination mirrors the source and understands when to use SCP vs SFTP vs rsync.

Note: Full commands and reference links are in labs/lab-13-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 14 — High Availability: Clustering, Load Balancing, NIC Teaming

Exam objective: 2.4 Clustering, fault tolerance, load balancing, NIC teaming.

Goal: The learner builds an HAProxy round-robin load balancer over two backends, observes failover and failback, and configures keepalived VRRP and NIC bonding.

What you'll build

An HAProxy load balancer with health checks plus keepalived and NIC-bond configs. (Tools: haproxy, keepalived, nginx, ifenslave.)

Step-by-step

115. Install the HA packages

```
apt update && apt install -y haproxy keepalived nginx ifenslave apache2-utils
```

116. Start two backend web servers on different ports

```
nohup python3 -m http.server 8001 --directory /srv/back1 >/tmp/b1.log 2>&1 &
```

117. Configure HAProxy round-robin with health checks

```
systemctl restart haproxy
```

118. Test round-robin distribution across backends

```
for i in 1 2 3 4; do curl -s 127.0.0.1:8080; done
```

119. Switch the algorithm to least-conn

```
sed -i 's/balance roundrobin/balance leastconn/' /etc/haproxy/haproxy.cfg
```

120. Kill a backend to observe failover

```
kill $(lsof -ti :8001) || pkill -f '8001'
```

121. Configure keepalived VRRP heartbeat for a floating VIP
122. Configure NIC bonding (active-backup vs 802.3ad LACP)

```
modprobe bonding mode=active-backup miimon=100
```

123. Review the cluster-aware patching procedure

Test it

The learner verifies success by seeing traffic alternate across backends and only the surviving backend answer after a failover.

Note: Full commands and reference links are in labs/lab-14-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 15 — Virtualization with KVM / QEMU

Exam objective: 2.5 Purpose and operation of virtualization.

Goal: The learner creates a thin-provisioned qcow2 disk, defines a VM with virt-install, inspects virtual networking and adds a second virtual switch.

What you'll build

A defined KVM VM with a qcow2 disk and a second isolated virtual switch. (Tools: qemu-kvm, libvirt-daemon-system, virtinst, bridge-utils.)

Step-by-step

124. Install the virtualization stack and start libvirtd

```
apt update && apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system virtinst bridge-utils
```

125. Check whether the host CPU supports hardware virt

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

126. Create a thin-provisioned qcow2 virtual disk

```
qemu-img create -f qcow2 /var/lib/libvirt/images/vm1.qcow2 5G
```

127. Define a VM mapping each flag to objective 2.5

```
virsh list --all
```

128. Inspect the default virtual network and virbr0

```
virsh net-dumpxml default
```

129. Compare bridged vs NAT vs isolated networking
130. Add and start a second virtual switch

```
virsh net-start internal
```

131. Review CPU/memory/disk/NIC overprovisioning rules

```
nproc
```

132. Compare public, private and hybrid cloud models

Test it

The learner verifies success by confirming the qcow2 disk is thin-provisioned and the second virtual network starts and appears in virsh net-list.

Note: Full commands and reference links are in labs/lab-15-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 16 — Scripting Basics for Server Administration

Exam objective: 2.6 Scripting (types, variables, loops, conditionals, admin tasks).

Goal: The learner writes Bash scripts covering variables, data types, conditionals, loops and real admin tasks like service management and bulk account creation.

What you'll build

A set of Bash scripts for service checks, bulk account creation and host bootstrap. (Tools: bash, coreutils, systemctl.)

Step-by-step

133. Review the four script types and their platforms
134. Write a script using variables, comments and environment exports

```
chmod +x 01-basics.sh && ./01-basics.sh
```

135. Use integers, strings and arrays

```
chmod +x 02-types.sh && ./02-types.sh
```

136. Write a disk-usage conditional with comparators

```
chmod +x 03-conditionals.sh && ./03-conditionals.sh
```

137. Write for and while loops

```
chmod +x 04-loops.sh && ./04-loops.sh
```

138. Write a service-management task script

```
chmod +x 05-services.sh && ./05-services.sh
```

139. Write a bulk account-creation task script

```
chmod +x 06-accounts.sh && ./06-accounts.sh
```

140. Write a bootstrap script with set -euo pipefail discipline

Test it

The learner verifies success by running each script and confirming the service, account and bootstrap tasks produce the expected output.

Note: Full commands and reference links are in labs/lab-16-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 17 — Asset Management, Documentation, and Licensing

Exam objective: 2.7/2.8 Asset management, documentation, licensing concepts.

Goal: The learner scripts asset discovery, captures an as-built config set, reviews BIA-driven metrics and the licensing model table, and validates a license count.

What you'll build

A discovered asset CSV, an as-built config snapshot and a license-count report. (Tools: dmidecode, coreutils, spreadsheet.)

Step-by-step

141. Auto-discover the asset details into a CSV
142. Review the asset life-cycle stages
143. Capture the running config as an as-built doc set

```
systemctl list-units --type=service --state=running > services.txt
```

144. Review BIA-driven metrics (MTBF, MTTR, RPO, RTO, SLA, uptime)
145. Draft a change-management entry template
146. Restrict permissions on sensitive documentation

```
chmod 750 /root/asbuilt
```

147. Review the Server+ licensing model cheat sheet
148. Run a license-count validation script for true-up

```
chmod +x /root/license-count.sh && /root/license-count.sh
```

Test it

The learner verifies success by producing the asset CSV, the as-built config files and a socket/core/vCPU count for a true-up.

Note: Full commands and reference links are in labs/lab-17-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Topic 03 — Security and Disaster Recovery (24%)

Data & physical security · IAM · mitigation & hardening · decommissioning · backup & DR

Key concepts

- Data securityEncryption at rest (LUKS/BitLocker) and in transit (TLS/SSH); retention policies and the data lifecycle.
- Physical securityAccess controls, mantraps, badge/biometric systems, cameras and environmental controls for the server room.
- Identity & access managementUser accounts, groups, RBAC, least privilege, MFA and permission management.
- Mitigation & hardeningMalware prevention, DLP, SIEM; OS updates, disabling unused services and host-based hardening.
- DecommissioningNIST 800-88 media sanitization — clear, purge, destroy; recycling and asset disposal records.
- Backup & disaster recoveryFull/incremental/differential backups, the 3-2-1 rule, snapshots, replication and site failover; RPO and RTO.

Lab 18 — Data at Rest (LUKS) and Data in Transit (TLS/SSH)

Exam objective: 3.1 Data security concepts (encryption at rest and in transit).

Goal: The learner encrypts a volume with LUKS, proves the ciphertext is unreadable, serves TLS with a self-signed cert and hardens SSH.

What you'll build

A LUKS-encrypted volume and a TLS-enabled nginx site with hardened SSH. (Tools: cryptsetup, openssl, openssh-server.)

Step-by-step

149. Install the encryption tools

```
apt update && apt install -y cryptsetup openssl openssh-server
```

150. Format a volume at rest with LUKS

```
cryptsetup luksFormat /dev/loop20 --batch-mode --key-file=<(echo -n
'LabPass!1')
```

151. Open, format and mount the encrypted volume

```
cryptsetup luksOpen /dev/loop20 secret --key-file=<(echo -n 'LabPass!1')
```

152. Close it and prove the raw bytes are unreadable

```
strings /srv/secret.img | grep financials || echo "no plaintext visible — at-rest encryption works"
```

153. Inspect the cipher and LUKS header

```
cryptsetup luksDump /dev/loop20 | head -30
```

154. Generate a self-signed TLS cert and serve HTTPS

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout /etc/ssl/private/lab.key -out /etc/ssl/certs/lab.crt -subj "/CN=lab.local" -days 365
```

155. Verify the negotiated TLS protocol and cipher

```
echo | openssl s_client -connect 127.0.0.1:443 -servername lab.local 2>/dev/null | grep -E "Protocol|Cipher\s+:"
```

156. Review SSH hardening lines and BIOS/GRUB passwords
157. Map a retention policy to data classification

Test it

The learner verifies success by confirming no plaintext is recoverable from the closed LUKS image and TLS 1.2/1.3 is negotiated.

Note: Full commands and reference links are in labs/lab-18-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 19 — Physical and Environmental Security Walk-Through

Exam objective: 3.2 Physical security concepts.

Goal: The learner inspects environmental sensors the OS can see and walks the physical access-control layers, lock types, environmental controls and a security checklist.

What you'll build

A completed physical-security audit checklist for a server room. (Tools: lm-sensors, smartmontools.)

Step-by-step

158. Install the sensor tools

```
apt update && apt install -y lm-sensors smartmontools
```

159. Review the physical access-control layers (defence in depth)
160. Review lock and authenticator types (RFID, biometric)
161. Inspect environmental sensors the OS exposes

```
sensors 2>/dev/null | head -30
```

162. Read drive temperature via SMART

```
smartctl -a /dev/sda 2>/dev/null | grep -i temperature | head -3
```

163. Review cameras and guards as detective controls
164. Map the concentric data-centre security zones
165. Review the mantrap defence against tailgating
166. Complete the physical-security audit checklist

Test it

The learner verifies success by reading available environmental sensor data and completing the physical-security checklist.

Note: Full commands and reference links are in labs/lab-19-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 20 — Identity & Access Management for Server Administration

Exam objective: 3.3 Identity and access management.

Goal: The learner builds RBAC groups with segregation of duties, delegates via sudoers, enforces password policy and lockout, and audits access.

What you'll build

Three RBAC roles with scoped sudoers rules, password policy and audit checks. (Tools: sudo, libpam-pwquality, chage, setfacl.)

Step-by-step

167. Install the IAM tools

```
apt update && apt install -y sudo libpam-pwquality
```

168. Create three role groups and users (segregation of duties)

```
useradd -m -G sysops alice && echo 'alice:AlicePass!1' | chpasswd
```

169. Delegate scoped commands per role via sudoers

```
visudo -c
```

170. Test that each role can only run its own commands

```
sudo -u bob   -i bash -c 'sudo systemctl restart mariadb && echo bob-ok'
```

171. Enforce password length and complexity policy
172. Enforce account lockout with pam_faillock
173. Enforce password aging

```
chage -M 90 -m 1 -W 7 alice
```

174. Layer file ACLs on top of group bits

```
setfacl -m g:netops:r-x /srv/dbdata
```

175. Audit logins, group membership and account changes

```
getent group sysops dbops netops
```

Test it

The learner verifies success by confirming a user is denied a command outside its role and password/lockout policy is enforced.

Note: Full commands and reference links are in labs/lab-20-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 21 — Multi-Factor Authentication with TOTP (PAM)

Exam objective: 3.3 Multifactor authentication.

Goal: The learner adds a TOTP second factor to SSH with PAM, provisions a user's secret, generates codes from the CLI and tests the two-factor login.

What you'll build

An SSH login enforcing a password plus a TOTP second factor. (Tools: libpam-google-authenticator, qrencode, oathtool.)

Step-by-step

176. Install the MFA tools and start ssh

```
apt update && apt install -y libpam-google-authenticator qrencode oathtool
openssh-server
```

177. Create a non-privileged user

```
useradd -m -s /bin/bash mfauser
```

178. Provision the TOTP secret for that user

```
sudo -u mfauser google-authenticator -t -d -f -r 3 -R 30 -w 17 -Q UTF8
```

179. Wire PAM and sshd to require TOTP

```
echo 'auth required pam_google_authenticator.so nullok' >
/etc/pam.d/sshd.mfa
```

180. Enable challenge-response and restart ssh

```
systemctl restart ssh
```

181. Generate the current TOTP from the CLI for testing

```
oathtool --totp -b "$SECRET"
```

182. Test the two-factor login

```
ssh -o PreferredAuthentications=keyboard-interactive,password -o
PubkeyAuthentication=no mfauser@127.0.0.1
```

183. Review scratch codes for account recovery

```
sudo tail -5 /home/mfauser/.google_authenticator
```

184. Review the factor-category test for true MFA

Test it

The learner verifies success by confirming login is denied with a wrong TOTP code even when the password is correct.

Note: Full commands and reference links are in labs/lab-21-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 22 — Auditing, Logging, and SIEM Basics

Exam objective: 3.4 Data security risks and mitigation (log analysis, SIEM).

Goal: The learner writes auditd rules for the four audit targets, configures rsyslog forwarding and rotation, runs logwatch and simulates a SIEM correlation rule.

What you'll build

Auditd file-watch rules, log rotation policy and a simulated correlation alert. (Tools: auditd, rsyslog, logwatch.)

Step-by-step

185. Install and enable auditd and rsyslog

```
apt update && apt install -y auditd audispd-plugins rsyslog logwatch
```

186. Write auditd rules for user/group/login/deletion events

```
augenrules --load
```

187. Trigger and search an audit event

```
ausearch -k id_changes -ts recent | tail -20
```

188. Review rsyslog severity levels and forwarding

```
logger -p auth.notice "Server+ lab test: notice-level entry"
```

189. Configure log rotation and retention

```
logrotate -d /etc/logrotate.d/srvplus 2>&1 | tail
```

190. Generate a daily report with logwatch

```
logwatch --output stdout --range today --detail Med | head -60
```

191. Simulate a SIEM correlation rule with awk

```
awk '/sshd.*Failed/ {fails[$NF]++} /sshd.*Accepted/ {if (fails[$NF]>=5)
print "ALERT: brute then success from", $NF}' /var/log/auth.log
```

192. Review two-person integrity and separation of roles

Test it

The learner verifies success by finding the triggered auditd event and confirming the correlation rule and rotation config work.

Note: Full commands and reference links are in labs/lab-22-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 23 — OS and Application Hardening

Exam objective: 3.5 Apply server hardening methods.

Goal: The learner baselines with Lynis, minimises services/ports/packages, hardens nginx, deploys AIDE and AppArmor, and re-scans to watch the hardening index climb.

What you'll build

A hardened host with a rising Lynis index, AIDE baseline and AppArmor enforcement. (Tools: lynis, apparmor-utils, aide, chkrootkit.)

Step-by-step

193. Install the hardening tools

```
apt update && apt install -y lynis apparmor-utils debsums aide chkrootkit
unattended-upgrades
```

194. Baseline the host with Lynis and note the hardening index

```
lynis audit system --quick 2>&1 | tail -60
```

195. Disable unused services

```
systemctl disable --now "$s"
```

196. Close unneeded ports with the firewall

```
ufw default deny incoming
```

197. Remove unneeded packages (minimisation)

```
apt purge -y telnet ftp rsh-client 2>/dev/null
```

198. Harden the nginx web role

```
nginx -t 2>/dev/null && systemctl reload nginx 2>/dev/null
```

199. Build an AIDE file-integrity baseline and check

```
aideinit -y -f 2>&1 | tail
```

200. Enforce an AppArmor profile (MAC)

```
aa-enforce /etc/apparmor.d/usr.sbin.nginx 2>/dev/null
```

201. Re-scan with Lynis and compare the score

```
lynis audit system --quick 2>&1 | grep -E "Hardening index|warning|
suggestion" | tail -20
```

Test it

The learner verifies success by confirming the Lynis hardening index improves after the minimisation and hardening steps.

Note: Full commands and reference links are in labs/lab-23-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 24 — Patch and Update Management

Exam objective: 3.5/2.4 Patching (testing, deployment, change management).

Goal: The learner distinguishes security from feature updates, snapshots and smoke-tests, applies patches with change discipline, and configures security-only auto-apply with a rollback plan.

What you'll build

A security-only unattended-upgrades config plus a patch report and rollback pin. (Tools: unattended-upgrades, needrestart, debsecan.)

Step-by-step

202. Install the patching tools

```
apt update && apt install -y unattended-upgrades needrestart apt-listchanges debsecan
```

203. Inventory current package versions as a baseline

```
apt list --upgradable 2>/dev/null | head -20
```

204. Identify security-only updates

```
apt-get -s dist-upgrade | grep -i security | head
```

205. Apply patches with change-management discipline

```
DEBIAN_FRONTEND=noninteractive apt -y -o Dpkg::Options::="--force-confdef" -o Dpkg::Options::="--force-confold" upgrade
```

206. Check which services need restarting after upgrade

```
needrestart -b
```

207. Check whether a reboot is required

```
[ -f /var/run/reboot-required ] && cat /var/run/reboot-required
```

208. Configure unattended-upgrades for security-only auto-apply

```
unattended-upgrade --dry-run -d 2>&1 | tail
```

209. Plan a rollback via package pinning

```
apt-mark hold nginx
```

210. Write a patch report for the change ticket

Test it

The learner verifies success by confirming the security-only auto-apply config validates and a rollback pin holds the package version.

Note: Full commands and reference links are in labs/lab-24-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 25 — Secure Decommissioning and Media Destruction

Exam objective: 3.6 Server decommissioning concepts.

Goal: The learner works the decommission checklist, wipes a loopback image with shred and nwipe, and reviews SSD erase, physical destruction and recycling.

What you'll build

A securely wiped media image with a documented decommission and destruction trail. (Tools: shred, wipe, nwipe, dd.)

Step-by-step

211. Install the wipe tools

```
apt update && apt install -y wipe nwipe secure-delete
```

212. Review the decommission workflow checklist
213. Create a drive image with sensitive content

```
strings /srv/oldserver.img | grep -E "pii|cred" | head
```

214. Logically wipe with shred (single pass)

```
shred -v -n 1 -z /srv/oldserver.img
```

215. Confirm no plaintext is recoverable

```
strings /srv/oldserver.img | grep -E "pii|cred" | head || echo "no
plaintext recoverable"
```

216. Multi-pass wipe with nwipe (DoD method)

```
nwipe --autonuke --nowait --method=dod /srv/oldserver.img 2>&1 | tail -10
```

217. Review SSD-specific ATA Secure Erase and crypto-erase
218. Review physical destruction methods (degauss, shred, crush, incinerate)
219. Review cable remediation and internal vs external recycling

Test it

The learner verifies success by confirming the sensitive strings are unrecoverable after the wipe.

Note: Full commands and reference links are in labs/lab-25-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 26 — Backup Strategy: Full, Incremental, Differential, Snapshot

Exam objective: 3.7 Backups and restores.

Goal: The learner runs full, incremental and differential backups with tar, a deduplicated synthetic full with restic, and validates a restore against the source.

What you'll build

A validated backup set spanning full, incremental, differential and restic snapshots. (Tools: tar, rsync, restic, lvm2.)

Step-by-step

220. Install the backup tools

```
apt update && apt install -y rsync restic lvm2
```

221. Prepare a live dataset and backup target

```
for i in $(seq 1 10); do dd if=/dev/urandom of=/srv/data/file$i.bin bs=1K
count=$((RANDOM%50+5)) 2>/dev/null; done
```

222. Take a full backup with tar

```
tar -czf /srv/backup/full-$(date +%F).tgz -C /srv data
```

223. Take an incremental backup with a snapshot file

```
tar --listed-incremental=/srv/backup/snap.snar -czf /srv/backup/inc-$(date
+%F).tgz -C /srv data
```

224. Take a differential backup

```
tar --listed-incremental=/srv/backup/snap-full.snar -czf /srv/backup/diff-$
(date +%F).tgz -C /srv data
```

225. Take deduplicated synthetic-full snapshots with restic

```
restic -r /srv/backup/restic backup /srv/data
```

226. Review media types and GFS rotation
227. Restore side-by-side and to an alternate location

```
restic -r /srv/backup/restic restore latest --target /srv/restore-alt
```

228. Validate integrity and diff the restore against source

```
restic -r /srv/backup/restic check
```

Test it

The learner verifies success by confirming the restic integrity check passes and the restored data matches the source.

Note: Full commands and reference links are in labs/lab-26-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 27 — Disaster Recovery: Replication and Site Failover

Exam objective: 3.8 Disaster recovery (site types, replication, testing).

Goal: The learner replicates data async with rsync and near-sync with lsyncd, reviews sync/DRBD concepts, plans keepalived VIP failover and runs a tabletop DR exercise.

What you'll build

An async and near-real-time replication pair plus a DR tabletop exercise plan. (Tools: rsync, lsyncd, keepalived, inotify-tools.)

Step-by-step

229. Install the DR tools

```
apt update && apt install -y rsync lsyncd keepalived inotify-tools
```

230. Review hot/warm/cold/cloud site types
231. Replicate asynchronously with rsync

```
rsync -avh --delete /srv/primary/ /srv/dr/
```

232. Configure near-real-time replication with lsyncd

```
systemctl restart lsyncd
```

233. Confirm the change propagated to the DR copy

```
cat /srv/dr/app.txt
```

234. Review synchronous replication (DRBD, semi-sync DB) trade-offs
235. Plan a keepalived VIP failover for DR cut-over
236. Review the four DR test types
237. Run the tabletop DR exercise and capture action items

Test it

The learner verifies success by confirming a change on the primary appears in the DR copy within seconds via lsyncd.

Note: Full commands and reference links are in labs/lab-27-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Topic 04 — Troubleshooting (28%)

Structured methodology · hardware · storage · OS/software · network · DR validation

Key concepts

- MethodologyThe CompTIA 6-step model: identify the problem, theorise, test, plan, implement & verify, document — change one thing at a time.
- Hardware faultsPower issues, POST/beep codes, failed drives (smartctl), overheating and memory errors (memtester, sensors).
- Storage faultsDegraded/failed arrays, full or corrupt file systems (fsck), I/O bottlenecks and open-file locks.
- OS & software faultsBoot and service failures (systemd, journalctl), failed patches, dependency and permission errors.
- Network faultsLatency, DNS and gateway misconfiguration, packet loss and cabling — diagnosed with ping, mtr, dig and tcpdump.
- DR validationTesting backups and restores, verifying failover, and confirming RPO/RTO are actually met.

Lab 28 — Troubleshooting Methodology Walk-Through

Exam objective: 4.1 Troubleshooting theory and methodology.

Goal: The learner breaks nginx, then applies the Server+ 7-step methodology end-to-end from identifying the problem through root-cause analysis and documentation.

What you'll build

A resolved incident with an RCA and a documented post-incident record. (Tools: nginx, journalctl, ss, systemctl.)

Step-by-step

238. Manufacture a fault by breaking the nginx listen port

```
sed -i 's/listen 80;/listen 22;/' /etc/nginx/sites-enabled/default
```

239. Identify the problem and scope by collecting logs

```
journalctl -u nginx -n 40 --no-pager
```

240. Replicate the fault from the server itself

```
curl -sI http://127.0.0.1/ 2>&1 | head
```

241. Back up the config before making changes

```
cp /etc/nginx/sites-enabled/default /root/default.pre-fix.$(date +%s)
```

242. Establish and test a theory (port 22 already in use)

```
ss -tlnp 'sport = :22'
```

243. Implement the solution, one change at a time

```
sed -i 's/listen 22;/listen 80;/' /etc/nginx/sites-enabled/default
```

244. Verify full system functionality including dependencies

```
curl -sI http://127.0.0.1/ | head -1
```

245. Add a preventive config-validation measure
246. Perform a 5-whys root-cause analysis and document it

Test it

The learner verifies success by confirming nginx returns HTTP 200 and produces a documented RCA record.

Note: Full commands and reference links are in labs/lab-28-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 29 — Troubleshooting Common Hardware Failures

Exam objective: 4.2 Troubleshoot common hardware failures.

Goal: The learner reads event logs first, stress-tests CPU thermals, checks memory/ECC and SMART predictive failures, and maps POST/LED/olfactory cues to causes.

What you'll build

A hardware fault decision tree exercised against event logs, sensors and SMART. (Tools: dmesg, smartmontools, lm-sensors, memtester.)

Step-by-step

247. Install the hardware diagnostic tools

```
apt update && apt install -y dmidecode lshw smartmontools lm-sensors
memtester stress-ng edac-utils
```

248. Check event logs first for hardware faults

```
journalctl -k | grep -iE 'mce|edac|temperature|throttl|over.?heat|ecc' |
tail
```

249. Stress-test CPU and read thermals/utilisation

```
stress-ng --cpu $(nproc) --timeout 10s --metrics-brief 2>&1 | tail
```

250. Check memory errors, ECC and crash-dump terms

```
dmesg | grep -i mce | tail
```

251. Read SMART predictive-failure attributes

```
smartctl -a /dev/sda 2>/dev/null | head -40
```

252. Review PSU/fan faults via the BMC SEL
253. Review CMOS battery failure symptoms

```
chronyc tracking 2>/dev/null | head
```

254. Map POST codes and visual/auditory/olfactory cues to causes
255. Distinguish host-hardware faults from misallocated-VM symptoms

```
iostat -x 1 3
```

Test it

The learner verifies success by mapping each Server+ 4.2 symptom to a concrete tool and reading SMART health for predictive failure.

Note: Full commands and reference links are in labs/lab-29-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 30 — Storage Troubleshooting

Exam objective: 4.3 Troubleshoot storage problems.

Goal: The learner diagnoses disk/inode-full and slow I/O, recovers a corrupt superblock, rebuilds a failed RAID drive, and reviews cache-battery and cable faults.

What you'll build

Recovered filesystems and a rebuilt RAID array with a storage-fault tool map. (Tools: mdadm, e2fsprogs, lsof, iotop.)

Step-by-step

256. Install the storage diagnostic tools

```
apt update && apt install -y mdadm lvm2 e2fsprogs xfsprogs parted gdisk
smartmontools lsof iotop sysstat
```

257. Check capacity and inode utilisation

```
df -i
```

258. Diagnose slow I/O with iostat and iotop

```
iostat -xz 1 5
```

259. Corrupt and then recover a filesystem via a backup superblock

```
fsck.ext4 -b 32768 $LOOP || true
```

260. Create a RAID 5 array and fail a drive

```
mdadm /dev/md30 --fail /dev/loop32 --remove /dev/loop32
```

261. Add a spare and watch the array rebuild

```
mdadm /dev/md30 --add /dev/loop35
```

262. Review cache-battery failure and write slowdown
263. Read OS clues for cable/connector/backplane faults

```
dmesg | grep -iE "ata|scsi|sas|link rate|hard reset" | tail
```

264. Diagnose page/swap errors

```
swapon --show
```

Test it

The learner verifies success by mounting the recovered filesystem and confirming the RAID array rebuilds in /proc/mdstat.

Note: Full commands and reference links are in labs/lab-30-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 31 — OS and Software Troubleshooting

Exam objective: 4.4 Troubleshoot common OS and software problems.

Goal: The learner triages systemd services, fixes clock skew and broken dpkg state, reviews recovery boot modes, and uses Ansible to prevent config drift.

What you'll build

A recovered service and package state plus an idempotent Ansible playbook. (Tools: systemd, strace, chrony, ansible.)

Step-by-step

265. Install the OS troubleshooting tools

```
apt update && apt install -y strace chrony ansible
```

266. Triage a service: status, dependencies, failed units

```
systemctl list-units --type=service --state=failed
```

267. Diagnose a hanging service with strace

```
strace -p $PID -e trace=accept4,read,write -c -f -t -o /tmp/strace.log &
```

268. Diagnose and fix clock skew that breaks auth

```
chronyc -a 'burst 4/4'
```

269. Recover from a broken dpkg/patch state

```
dpkg --configure -a 2>&1 | head
```

270. Check missing dependencies and downstream failures

```
apt-cache rdepends openssl | head
```

271. Review recovery boot modes (rescue, emergency, snapshot)
272. Diagnose memory leak and set CPU affinity/priority

```
ps -eo pid,comm,rss,nice,psr --sort=-rss | head -10
```

273. Prevent config drift with an idempotent Ansible playbook

```
ansible-playbook playbook.yml 2>&1 | tail -10
```

Test it

The learner verifies success by confirming the service and package state recover and the Ansible playbook reports no changes on a second run.

Note: Full commands and reference links are in labs/lab-31-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 32 — Network Connectivity Troubleshooting

Exam objective: 4.5 Troubleshoot network connectivity issues.

Goal: The learner triages 'I can't reach the server' bottom-up through the OSI layers using ip, ping, traceroute, dig, nc and tcpdump.

What you'll build

A repeatable bottom-up network triage exercised layer by layer. (Tools: iproute2, dnsutils, tcpdump, mtr-tiny.)

Step-by-step

274. Install the network diagnostic tools

```
apt update && apt install -y iproute2 iputils-ping dnsutils tcpdump mtr-tiny traceroute nmap netcat-openbsd
```

275. Layer 1: check link, cable and NIC state

```
ethtool eth0 2>/dev/null | head
```

276. Layer 2: check NIC config and the ARP table

```
ip neigh
```

277. Layer 3: check addressing, gateway and routes

```
ip route get 8.8.8.8
```

278. Test reachability with ping, traceroute and mtr

```
mtr -rwc 5 8.8.8.8 2>/dev/null | head
```

279. Resolve names down the DNS chain

```
dig +short example.com
```

280. Test port reachability with nc and nmap

```
nc -vz example.com 80 2>&1 | head
```

281. Capture packets to find the silent direction

```
tcpdump -ni eth0 -c 10 'icmp or port 53' 2>&1 | head
```

282. Work the bottom-up decision tree

Test it

The learner verifies success by walking a connectivity failure layer by layer and pinpointing the drop with tcpdump.

Note: Full commands and reference links are in labs/lab-32-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Lab 33 — Security Troubleshooting

Exam objective: 4.6 Troubleshoot security problems.

Goal: The learner audits open ports and process states, checks file integrity with AIDE, hunts privilege-escalation misconfigs and runs an anti-malware scan.

What you'll build

A security triage covering ports, processes, file integrity and malware. (Tools: nmap, lsof, aide, chkrootkit, clamav.)

Step-by-step

283. Install the security troubleshooting tools

```
apt update && apt install -y nmap lsof aide chkrootkit clamav clamav-daemon auditd
```

284. Audit open ports against the firewall policy

```
nmap -sT -p- -T4 127.0.0.1 2>/dev/null | head -30
```

285. Classify process states: active, orphan, zombie

```
ps -eo pid,ppid,state,user,comm | awk '$3=="Z"'
```

286. Scan for rogue processes and rootkits

```
chkrootkit | head -30
```

287. Baseline and check file integrity with AIDE

```
aide --check 2>&1 | head
```

288. Hunt world-writable, SUID and writable-sudoers misconfigs

```
find / -xdev \( -perm -4000 -o -perm -2000 \) -type f 2>/dev/null | head
```

289. Run an anti-malware scan with the EICAR test file

```
clamscan /tmp/scan/ 2>&1 | tail -10
```

290. Triage 'cannot open file': perms, MAC policy, service health

```
getfacl /srv/share/
```

291. Work the security decision tree

Test it

The learner verifies success by detecting the AIDE integrity drift and the EICAR test file with ClamAV.

Note: Full commands and reference links are in labs/lab-33-*.md. Run every lab inside the disposable Killercoda VM; only run scanning, password or media-destruction techniques against systems you own or are authorised to test.

Exam Focus — Cross-Cutting Server+ Topics

These cross-cutting topics are examined throughout SK0-005 but are woven across several labs rather than each having a single dedicated one. Study this section alongside the labs so you can answer the knowledge questions on the exam.

RAID capacity and fault-tolerance (Domain 1)

Know the usable-capacity and fault-tolerance formula for every RAID level — the exam tests these directly:

- RAID 0 (stripe) — usable = N x disk; tolerates 0 drive failures; performance only, no redundancy.
- RAID 1 (mirror) — usable = 1 x disk (50% overhead); tolerates 1 failure; simple redundancy for the OS disk.
- RAID 5 (single parity) — usable = (N - 1) x disk; tolerates 1 failure; poor for write-heavy workloads.
- RAID 6 (double parity) — usable = (N - 2) x disk; tolerates 2 failures; preferred for large SATA arrays with long rebuilds.
- RAID 10 (stripe of mirrors) — usable = N / 2; tolerates 1 failure per mirror; best blend of speed and redundancy for databases.
- DAS vs. NAS vs. SAN; NFS/SMB serve files, iSCSI/Fibre Channel serve block LUNs; always plan capacity with growth headroom.

Networking and subnetting (Domain 2)

- IPv4 addressing, CIDR and subnet masks; work these out with the free IP Calculator at https://alfredang.github.io/ipcalculator/.
- DNS record types (A, AAAA, PTR, CNAME, MX), DHCP scopes and reservations, FQDN and the default gateway.
- VLANs segment a switch into broadcast domains; a server trunk carries tagged VLANs (802.1Q).
- Virtualization: host vs. guest, type-1 vs. type-2 hypervisors, resource allocation and over-commit; IaaS/PaaS/SaaS and public/private/hybrid cloud models.

Security, hardening and the data lifecycle (Domain 3)

- Encryption at rest (LUKS, BitLocker) vs. in transit (TLS, SSH); manage keys and certificates carefully.
- IAM — least privilege, RBAC, strong password policy and MFA (TOTP); disable unused accounts.
- Hardening — patch promptly, disable unused services and ports, apply host firewalls and MAC (AppArmor/SELinux), and audit with a tool such as Lynis.
- Decommissioning follows NIST SP 800-88 — clear, purge or destroy media according to its sensitivity, and record the disposal.
- Backup types (full, incremental, differential, snapshot), the 3-2-1 rule, and DR metrics RPO (data loss tolerated) and RTO (time to recover).

The troubleshooting methodology (Domain 4)

Apply the CompTIA six-step model to every fault, and change only one thing at a time so you can attribute the fix:

- 1. Identify the problem — gather information, question users, note recent changes.
- 2. Establish a theory of probable cause (question the obvious first).
- 3. Test the theory to determine the cause; if it fails, form a new theory or escalate.
- 4. Establish a plan of action and identify potential effects.
- 5. Implement the solution (or escalate), then verify full system functionality.
- 6. Document findings, actions and outcomes.

Exam Preparation

- First pass: complete every lab on the Killercoda Ubuntu playground, reading the reference links in each lab file.
- Second pass: redo the labs from memory until the command workflow and flags are automatic.
- Review the 'Test it' check and the 'What you learned' bullets for any topic you find hard.
- Memorise the RAID capacity/fault-tolerance formulas, subnetting, and the six-step troubleshooting model.
- Sharpen exam readiness with the Tertiary Infotech CompTIA Server+ practice exam: https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-server-plus
- Take the free CompTIA practice assessment for SK0-005 and sit the exam via a Pearson VUE test centre or online proctoring.

Glossary

- **RAID —** Redundant Array of Independent Disks — combines drives for performance and/or fault tolerance (levels 0/1/5/6/10).

- **JBOD —** Just a Bunch Of Disks — disks concatenated into one volume with no striping or parity.
- **LVM —** Logical Volume Manager — abstracts physical disks into flexible, resizable logical volumes.
- **DAS / NAS / SAN —** Direct-attached / network-attached (file) / storage-area network (block) storage architectures.
- **iSCSI —** A protocol that carries SCSI block storage (LUNs) over an IP network.
- **Out-of-band management —** Managing a server independently of its OS via a BMC — IPMI, iDRAC or iLO.
- **Hypervisor —** Software that runs virtual machines; type-1 runs on bare metal, type-2 on a host OS.
- **VLAN —** Virtual LAN — a logically isolated broadcast domain on a shared physical switch (802.1Q).
- **High availability —** Removing single points of failure through clustering, load balancing and failover.
- **RPO / RTO —** Recovery Point Objective (acceptable data loss) / Recovery Time Objective (acceptable downtime).
- **3-2-1 rule —** Three copies of data, on two media types, with one copy off-site.
- **Hardening —** Reducing a server's attack surface by patching, disabling unused services and applying host controls.
- **NIST 800-88 —** The standard for media sanitization — clear, purge or destroy data before disposal.